Heart Disease Risk Prediction

End-to-End MLOps Project

Student Name	Rupesh Jha
Student ID	2025ae05964
Course	BITS Pilani — AIMLCZG523
Assignment	Assignment 01
Date	20 October 2026

A reproducible machine-learning workflow covering data preparation, model training, experiment tracking, API serving, containerization, orchestration and monitoring.

1. Setup and Installation

Project environment and execution workflow

The project implements a reproducible workflow for training and serving a binary heart-disease classification model. The following commands describe the main setup and execution steps used in the project.

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python src/data_download.py
python src/eda.py
python src/train.py
pytest -q
flake8
docker build -t heart-api:latest .
```

Core dependencies

• pandas and NumPy for data handling.

• scikit-learn for preprocessing, model training and evaluation.

• MLflow for experiment tracking and artifacts.

• pytest and flake8 for testing and code-quality checks.

• FastAPI for prediction serving; Docker and Kubernetes for deployment.

• Prometheus and Grafana for service monitoring and visualization.

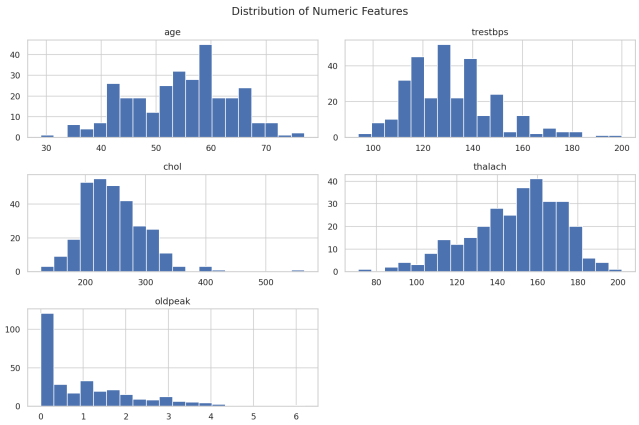
The downloaded dataset was saved to data/heart_raw.csv. The training pipeline saves the selected model to models/model.joblib and records experiment information in MLflow.

Repository: https://github.com/2025ae05964/mlops-heart-disease-prediction

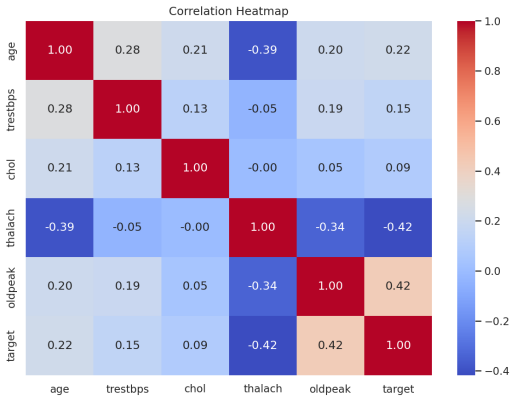
2. Dataset and Exploratory Data Analysis

The downloaded heart-disease dataset contains 303 records and 14 columns: 13 input features and the original target column. The preprocessing code converts the original target into a binary label: 0 means absence and values greater than 0 mean presence. Missing numeric values are imputed with the median and missing categorical values with the most frequent category.

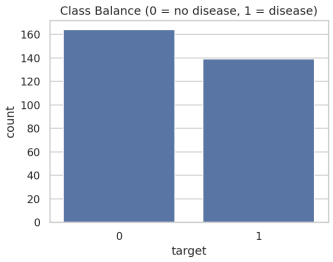
Feature distributions



Correlation heatmap



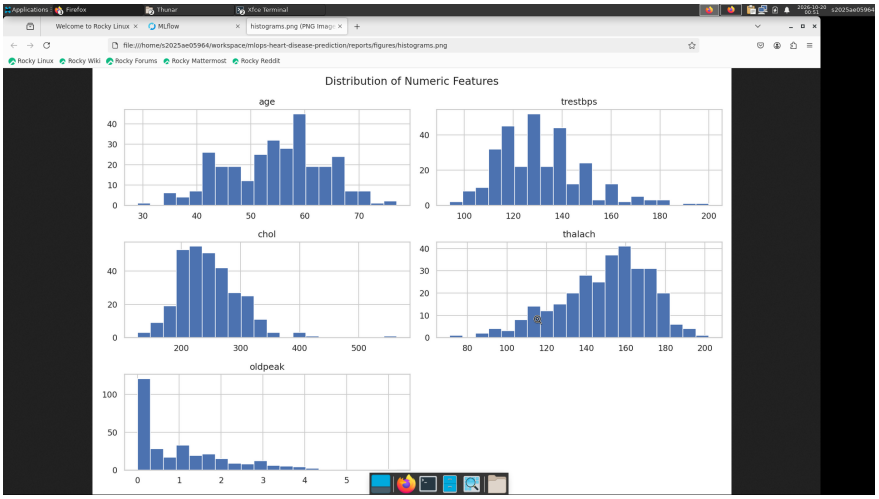
Target class balance



Observed class counts: 164 samples in class 0 and 139 samples in class 1.

3. EDA Findings and Interpretation

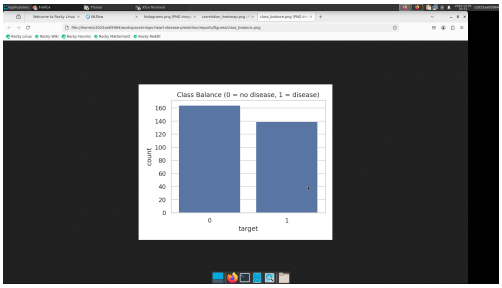
The exploratory analysis provides a view of feature distributions, relationships between variables and the balance of the prediction target.



Histograms show the spread and shape of individual features. Differences in ranges and distributions help identify why preprocessing is useful before model fitting.



The correlation heatmap summarizes pairwise linear relationships. Chest-pain type (cp), maximum heart rate (thalach) and ST depression (oldpeak) are useful variables to examine in relation to the target; correlation alone does not establish causation.



The classes are reasonably close in size, although class 0 has 25 more records than class 1. The dataset is not perfectly balanced, so precision and recall are reported alongside accuracy.

4. Feature Engineering and Model Selection

The preprocessing is defined as a scikit-learn ColumnTransformer and is fitted within each model pipeline, avoiding fitting transformations on the held-out test data.

Preprocessing steps

• Numeric features: median imputation followed by StandardScaler.

• Categorical features: most-frequent imputation followed by OneHotEncoder(handle_unknown='ignore').

• Target binarization: original target 0 becomes 0; original values 1–4 become 1.

• A train/test split is used for evaluation.

Models and tuning

• Logistic Regression: grid search over C = 0.1, 1.0 and 10.0; max_iter = 1000.

• Random Forest: grid search over 100 or 300 trees and max_depth = None, 5 or 10.

• GridSearchCV uses 5-fold cross-validation with ROC-AUC as its scoring metric.

• Cross-validation accuracy is also recorded; the final model is selected by test ROC-AUC.

Evaluation results

Metric	Logistic Regression	Random Forest
Accuracy	0.8852	0.8689
Precision	0.8387	0.8333
Recall	0.9286	0.8929
ROC-AUC	0.9654	0.9481
Mean CV accuracy	0.8346	0.8056

Logistic Regression achieved the higher ROC-AUC (0.9654) and accuracy (0.8852) in the recorded run.

5. Experiment Tracking with MLflow

The MLflow experiment is named heart-disease-classification. Training runs log model parameters, evaluation metrics and artifacts to support reproducibility and comparison.

Tracked information includes the tuned parameters, accuracy, precision, recall, ROC-AUC, cross-validation accuracy, saved model artifacts and EDA plots.

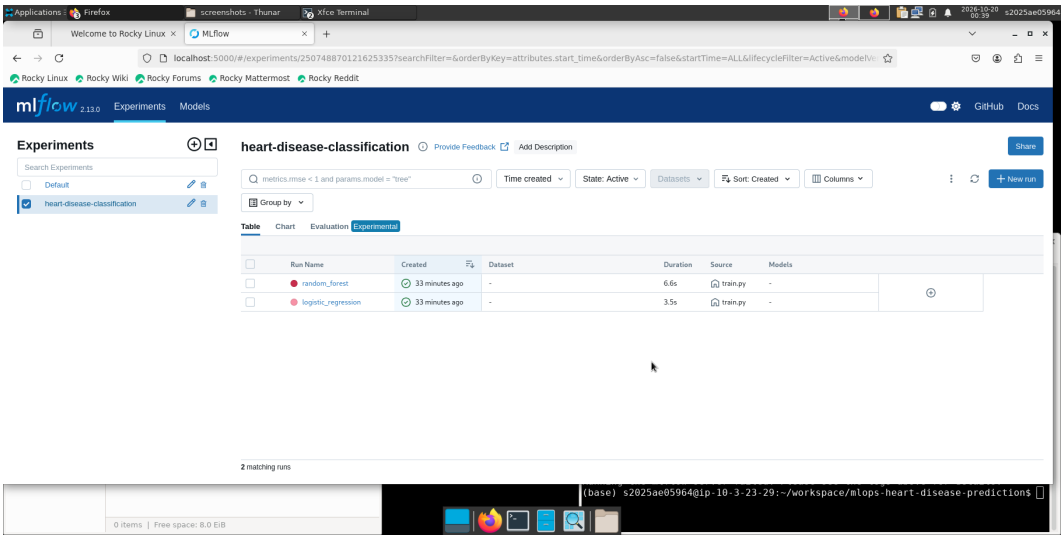


Figure: MLflow experiment runs.

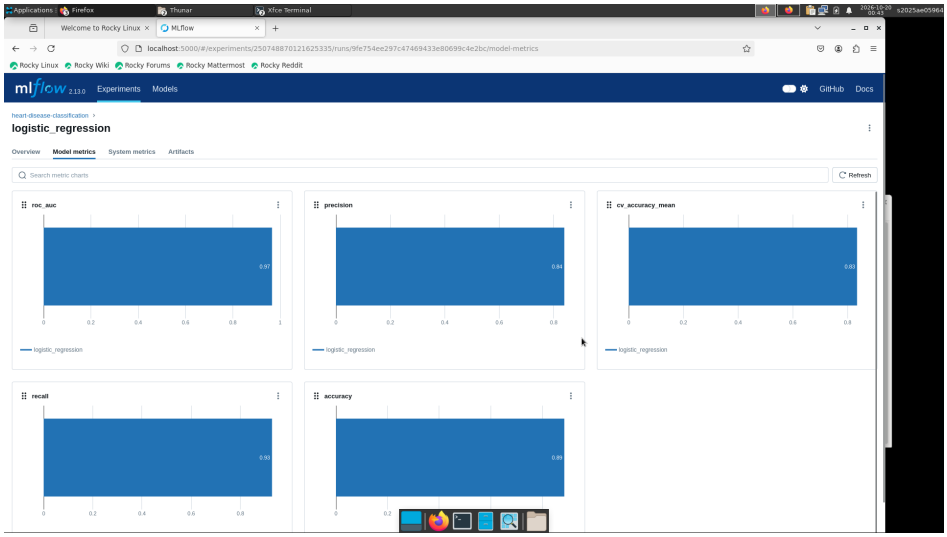


Figure: Logged model metrics for comparison.

The selected model is serialized to models/model.joblib and the model artifact is also logged with MLflow.

6. Testing, Code Quality and CI/CD

Automated checks help catch regressions and keep the training and serving code maintainable.

Validation evidence

• pytest -q completed with 3 tests passing.

• flake8 was run to check code style.

• The training workflow logs its parameters, metrics and model artifacts.

• The GitHub repository contains the project source and workflow configuration.

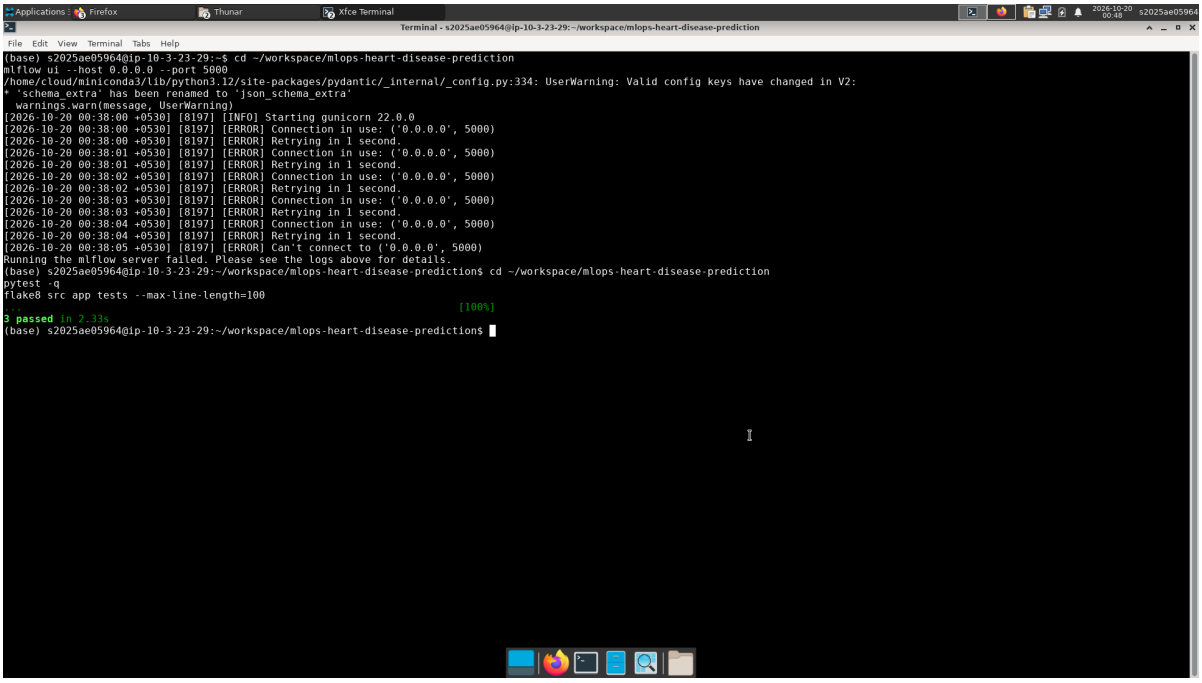
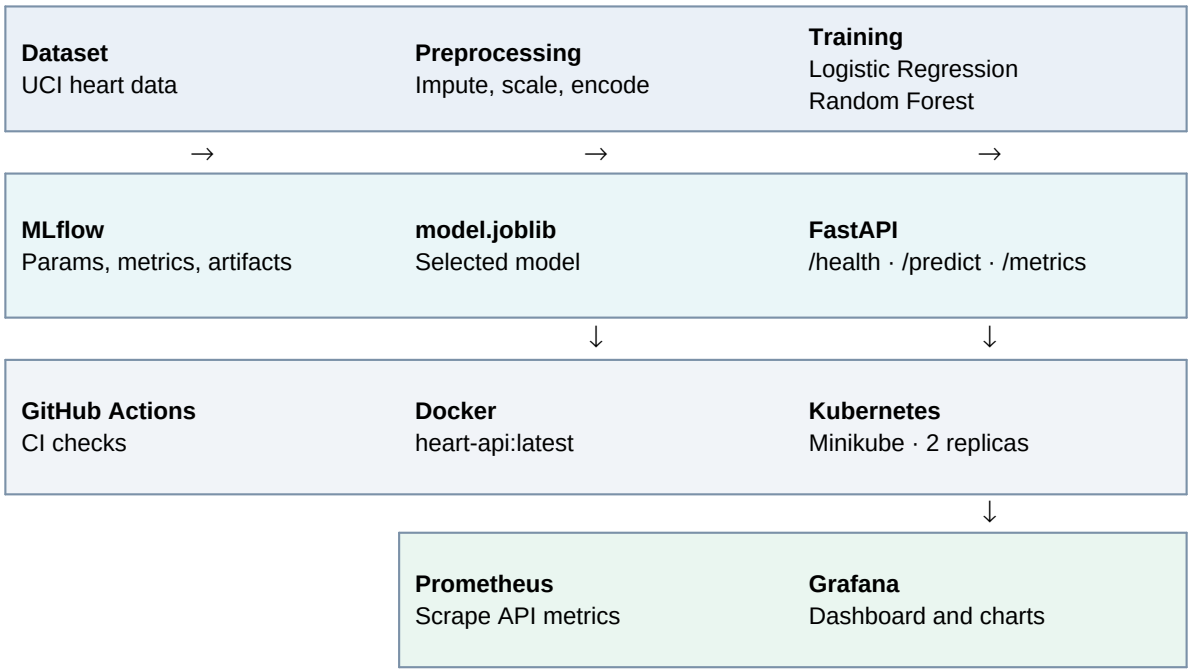


Figure: Test and lint command output.

The CI/CD workflow is intended to automate code-quality and test checks when changes are pushed. This report includes the locally captured test evidence; a separate GitHub Actions success screenshot was not present in the available screenshots folder.

7. End-to-End System Architecture

The diagram summarizes the data-to-model workflow and the deployed prediction service.



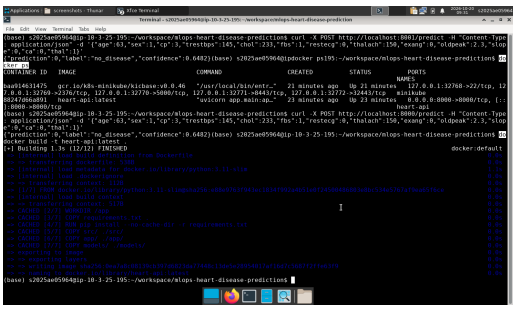
Request path: client → FastAPI prediction endpoint → loaded model pipeline → JSON prediction. The monitoring path exposes Prometheus metrics for visualization in Grafana.

Deployment evidence is documented on the following page.

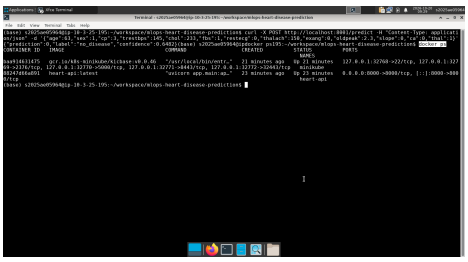
8. Containerization and Kubernetes Deployment

The application was packaged into the Docker image heart-api:latest. The API exposes a health endpoint and a prediction endpoint. The model returned a valid JSON response in both Docker and Kubernetes testing.

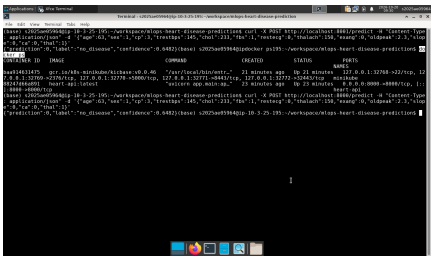
Docker image build



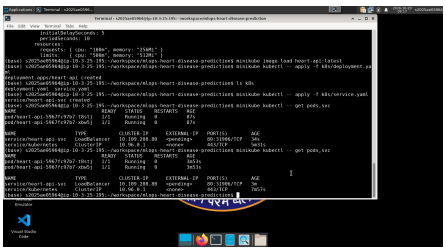
Running container



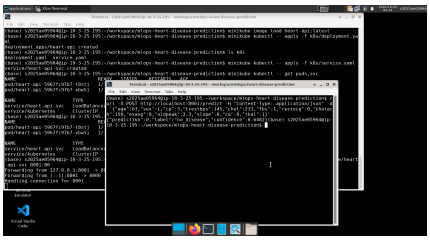
Prediction response



Kubernetes pods



Kubernetes prediction

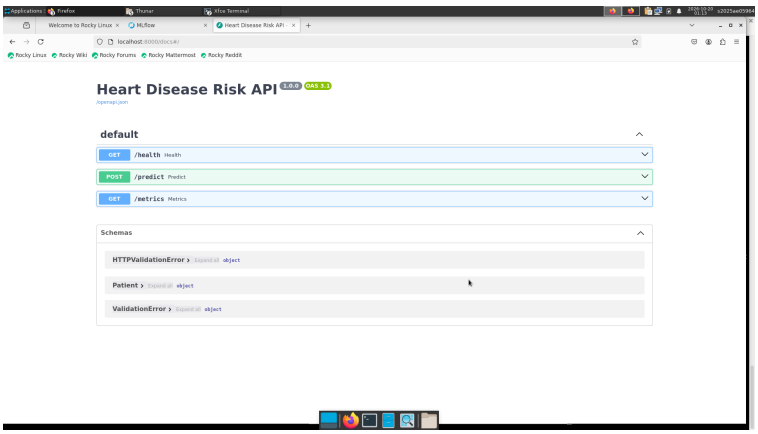


The Minikube deployment used two API replicas. The service was tested through port forwarding; a LoadBalancer external IP may remain pending in a local Minikube environment.

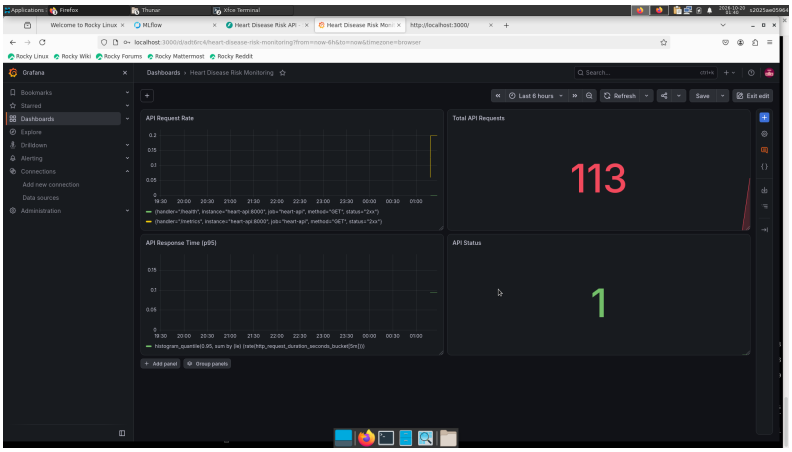
9. Monitoring, API Validation and Conclusion

The service exposes GET /health, POST /predict and GET /metrics. The health response confirmed that the model was loaded, while the prediction endpoint returned a class label and confidence value.

API documentation



Monitoring dashboard



Prometheus metrics are used by the Grafana dashboard to display request activity, total requests, response-time information and API status.

Conclusion

This project demonstrates an end-to-end MLOps lifecycle: dataset acquisition and EDA, preprocessing within model pipelines, model comparison and tuning, MLflow experiment tracking, automated tests, API serving, Docker packaging, Kubernetes deployment and monitoring. In the recorded evaluation, Logistic Regression achieved ROC-AUC 0.9654 and accuracy 0.8852. These are dataset-specific evaluation results and should not be interpreted as a clinical diagnosis or proof of real-world medical performance.

GitHub repository

https://github.com/2025ae05964/mlops-heart-disease-prediction